Основы проектирования

Класс

Во втором томе книги Лутца “Изучаем Python” дается следующее определение классу в Python:

> **Класс Python** – кодовая структура и механизм, используемые для реализации в Python новых видов объектов, которые поддерживают наследование

Класс создается с помощью ключевого слова `class`:

```python
class Test:
    pass
```

Хотя он и кажется пустым, тем не менее он имеет целую охапку полей и методов, которые python создает автоматически. Их можно увидеть с помощью функции `dir()` и `var()`:

```python
dir(Test)
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__', '__forma
```

```python
vars(Test)
```

```
mappingproxy({'__module__': '__main__', '__firstlineno__': 1, '__static_attributes__': (), '__dict__
```

Чтобы создать объект класса достаточно вызвать его с синтаксисом похожим на функцию

```python
a = Test()
a
```

```
<__main__.Test object at 0x1046ce510>
```

На самом деле это и является неявно вызываемыми функциями, которая вызывается каждый раз при создании объекта. А конкретно метод `__new__` класса Test (выделяет память и создает объект данного класса). А так же метод `__init__` объекта, который дает логику внутрь объекта (к примеру заполняет его поля). Так что семантически такой вариант будет аналогичным:

```python
a = Test.__new__(Test)

if isinstance(a, Test):
    a.__init__()

a
```

```
<__main__.Test object at 0x10471cf50>
```

Так что если при написании класса мы определим метод `__init__`, тогда объект класса будет иметь соответсвующую логику:

```python
class Test:
    def __init__(self):
```

```
        print("Hello, world!")
        self.a = 123
```

```
a = Test()
```
Python

```
Hello, world!
```

```
a.a
```
Python

```
123
```

Аналогично:

```
b = Test.__new__(Test)
```
Python

```
if isinstance(a, Test):
    b.__init__()
```

```
Hello, world!
```

Python

```
b.a
```

```
123
```

Так же мы можем в рантайме изменять, добавлять и удалять атрибуты и методы

```
b.b = "Новый аттрибут"
b.b
```
Python

```
'Новый аттрибут'
```

```
def func(num):
    print(f"Новый метод №{num}")

b.c = func
b.c(1)
```
Python

```
Новый метод №1
```

При этом вызвать новый метод будет именно методом конкретного объекта:

```
a.b
a.c(2)
```
Python

```
Traceback (most recent call last):
  File "<chunk>", line 1, in <module>
AttributeError: 'Test' object has no attribute 'b'
```

Такой подход позволяет реализовывать активную интроспекцию и рефлексию, меняя, добавляя и удаляя поля во время работы программы.

Что внутри класса

Python сильно более объектно ориентирован, чем многие языки. Почти все что вы встретите в python будет объектом.
Функции, числа, классы, модули - это все объекты.

По сути тип данных в python - это класс объекта. Так что для получения класса объекта есть `type`

```python
type(1)
```

```
<class 'int'>
```

```python
type([1, "2", (3,)])
```

```
<class 'list'>
```

```python
class A: ...
type(A())
```

```
<class '__main__.A'>
```

Но в python как говорилось раньше практически все - это объект К примеру:

```python
type(A)
```

```
<class 'type'>
```

```python
type(type)
```

```
<class 'type'>
```

```python
def func(): ...
type(func)
```

```
<class 'function'>
```

```python
type(type(func))
```

```
<class 'type'>
```

На уровне реализации языка внутри каждый объект - это struct в Си, которая содержит:

- Счетчик ссылок: `ob_refcnt`
- Указатель на тип: `ob_type`

И выглядит примерно так:

```c
typedef struct _object {
    _PyObject_HEAD_EXTRA
    Py_ssize_t ob_refcnt;
    PyTypeObject *ob_type;
} PyObject;
```

Тут `_PyObject_HEAD_EXTRA` - макрос, который добавляет указатели на предыдущий и следующий объект в памяти. Это позволяет сборщику мусора ходить по двухсвязному списку всех активных объектов и удалять их в случае необходимости

Для объектов переменной длины так же имеются поле `ob_size`. Такие объекты называются `PyVarObject`.

```c
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_size;
} PyVarObject;
```

Интерфейс

Protocol

Метаклассы

Метаклассы в Python очень обширная и сложная тема. Почти всегда когда есть возможность избежать использования метаклассов в проектировании системы лучше этого избегать.

По сути метаклассы - это классы объектами которыми являются другие классы.
Раньше мы применяли `type` как функцию, которая принимает объект и возвращает класс объекта.
Но на самом деле это класс, который при передаче одного аргумента возвращает класс, к которому относится объект

```python
class MySuperClass: ...
class MyMixin: ...

class MyClass(MySuperClass, MyMixin):
    x = 42
    def x2(self):
        return self.x * 2

MyClass()
```

```
<__main__.MyClass object at 0x1046ce660>
```

С помощью конструктора `type` мы можем создать `MyClass` во время выполнения:

```python
MyClass = type('MyClass',
        (MySuperClass, MyMixin),
        {'x': 42, 'x2': lambda self: self.x * 2},
    )

MyClass()
```

```
<__main__.MyClass object at 0x1046ce660>
```